

multicollinearity__exercise

September 25, 2025

```
[1]: import numpy as np
import pandas as pd
from statsmodels.formula.api import ols
from plotnine import ggplot, aes, geom_point, labs
from scipy.stats import linregress
```

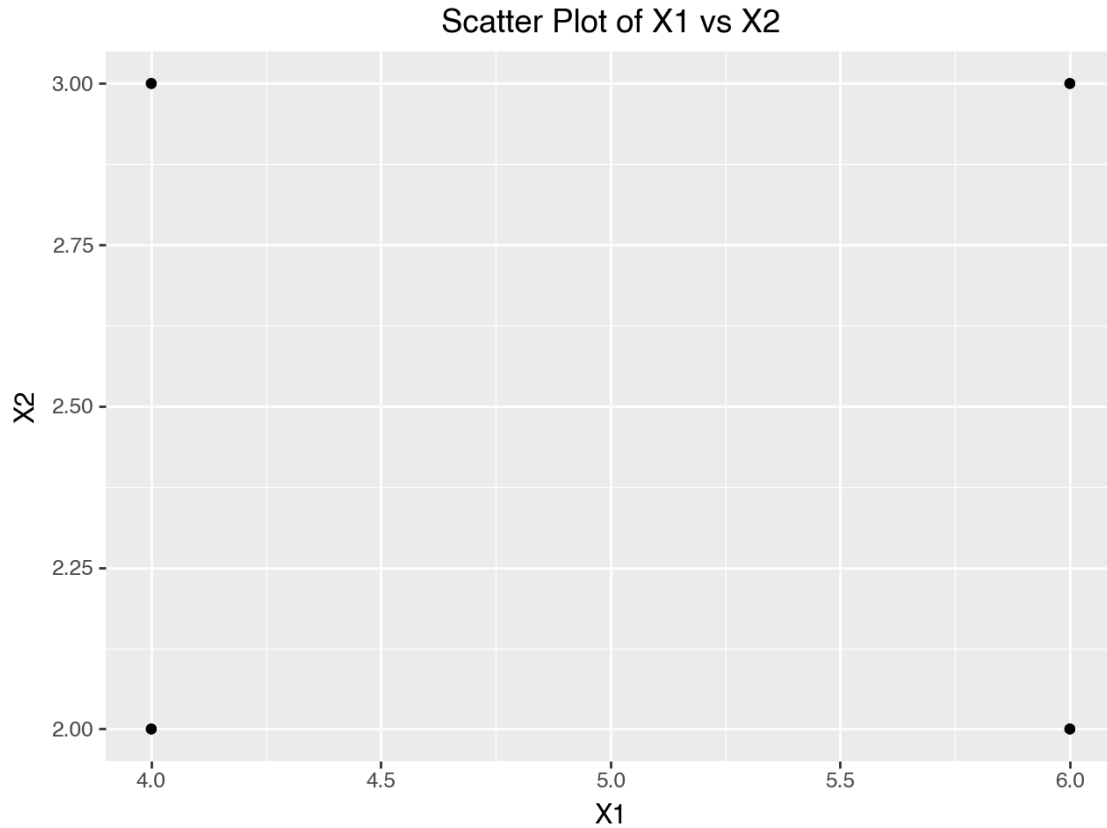
1 Multicollinearity Demo

```
[2]: ### multicollinearity

# Uncorrelated predictor variables
X1 = [4, 4, 4, 4, 6, 6, 6, 6]
X2 = [2, 2, 3, 3, 2, 2, 3, 3]
Y = [42, 39, 48, 51, 49, 53, 61, 60]
```

```
[3]: # Creating DataFrame
df = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'Y': Y
})

# Plot X1 vs X2
plot = (ggplot(df, aes(x='X1', y='X2')) +
        geom_point() +
        labs(x='X1', y='X2', title='Scatter Plot of X1 vs X2'))
plot
```



```
[4]: # Correlation between X1 and X2
correlation = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation}')
```

Correlation between X1 and X2: 0.0

```
[5]: # Linear model Y ~ X1 + X2
model1 = ols('Y ~ X1 + X2', data=df).fit()
model1.summary()
```

/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-packages/scipy/stats/_axis_nan_policy.py:531: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=8

```
[5]:
```

Dep. Variable:	Y	R-squared:	0.958
Model:	OLS	Adj. R-squared:	0.941
Method:	Least Squares	F-statistic:	57.06
Date:	Thu, 25 Sep 2025	Prob (F-statistic):	0.000361
Time:	10:51:30	Log-Likelihood:	-14.511
No. Observations:	8	AIC:	35.02
Df Residuals:	5	BIC:	35.26
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.3750	4.740	0.079	0.940	-11.811	12.561
X1	5.3750	0.664	8.097	0.000	3.669	7.081
X2	9.2500	1.328	6.968	0.001	5.837	12.663

Omnibus:	2.902	Durbin-Watson:	2.773
Prob(Omnibus):	0.234	Jarque-Bera (JB):	0.878
Skew:	-0.108	Prob(JB):	0.645
Kurtosis:	1.391	Cond. No.	42.1

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[6]: # Linear model Y ~ X1
model_X1 = ols('Y ~ X1', data=df).fit()
model_X1.summary()
```

/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-packages/scipy/stats/_axis_nan_policy.py:531: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=8

[6]:

Dep. Variable:	Y	R-squared:	0.550
Model:	OLS	Adj. R-squared:	0.476
Method:	Least Squares	F-statistic:	7.347
Date:	Thu, 25 Sep 2025	Prob (F-statistic):	0.0351
Time:	10:51:30	Log-Likelihood:	-23.995
No. Observations:	8	AIC:	51.99
Df Residuals:	6	BIC:	52.15
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	23.5000	10.111	2.324	0.059	-1.242	48.242
X1	5.3750	1.983	2.711	0.035	0.523	10.227

Omnibus:	3.389	Durbin-Watson:	1.815
Prob(Omnibus):	0.184	Jarque-Bera (JB):	0.935
Skew:	-0.117	Prob(JB):	0.626
Kurtosis:	1.341	Cond. No.	27.0

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[7]: # Linear model Y ~ X2
model_X2 = ols('Y ~ X2', data=df).fit()
model_X2.summary()
```

/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-packages/scipy/stats/_axis_nan_policy.py:531: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=8

```
[7]:
```

Dep. Variable:	Y	R-squared:	0.408
Model:	OLS	Adj. R-squared:	0.309
Method:	Least Squares	F-statistic:	4.128
Date:	Thu, 25 Sep 2025	Prob (F-statistic):	0.0885
Time:	10:51:30	Log-Likelihood:	-25.100
No. Observations:	8	AIC:	54.20
Df Residuals:	6	BIC:	54.36
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	27.2500	11.608	2.348	0.057	-1.153	55.653
X2	9.2500	4.553	2.032	0.088	-1.891	20.391

Omnibus:	4.296	Durbin-Watson:	0.359
Prob(Omnibus):	0.117	Jarque-Bera (JB):	1.012
Skew:	-0.008	Prob(JB):	0.603
Kurtosis:	1.257	Cond. No.	14.9

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[8]: # ANOVA for both models
from statsmodels.stats.anova import anova_lm

anova_results1 = anova_lm(model1)
anova_results1
```

```
[8]:
```

	df	sum_sq	mean_sq	F	PR(>F)
X1	1.0	231.125	231.125	65.567376	0.000466
X2	1.0	171.125	171.125	48.546099	0.000937
Residual	5.0	17.625	3.525	NaN	NaN

```
[9]: anova_results_X1 = anova_lm(model_X1)
anova_results_X1
```

```
[9]:
```

	df	sum_sq	mean_sq	F	PR(>F)
X1	1.0	231.125	231.125000	7.34702	0.035081
Residual	6.0	188.750	31.458333	NaN	NaN

```
[10]: anova_results_X2 = anova_lm(model_X2)
      anova_results_X2
```

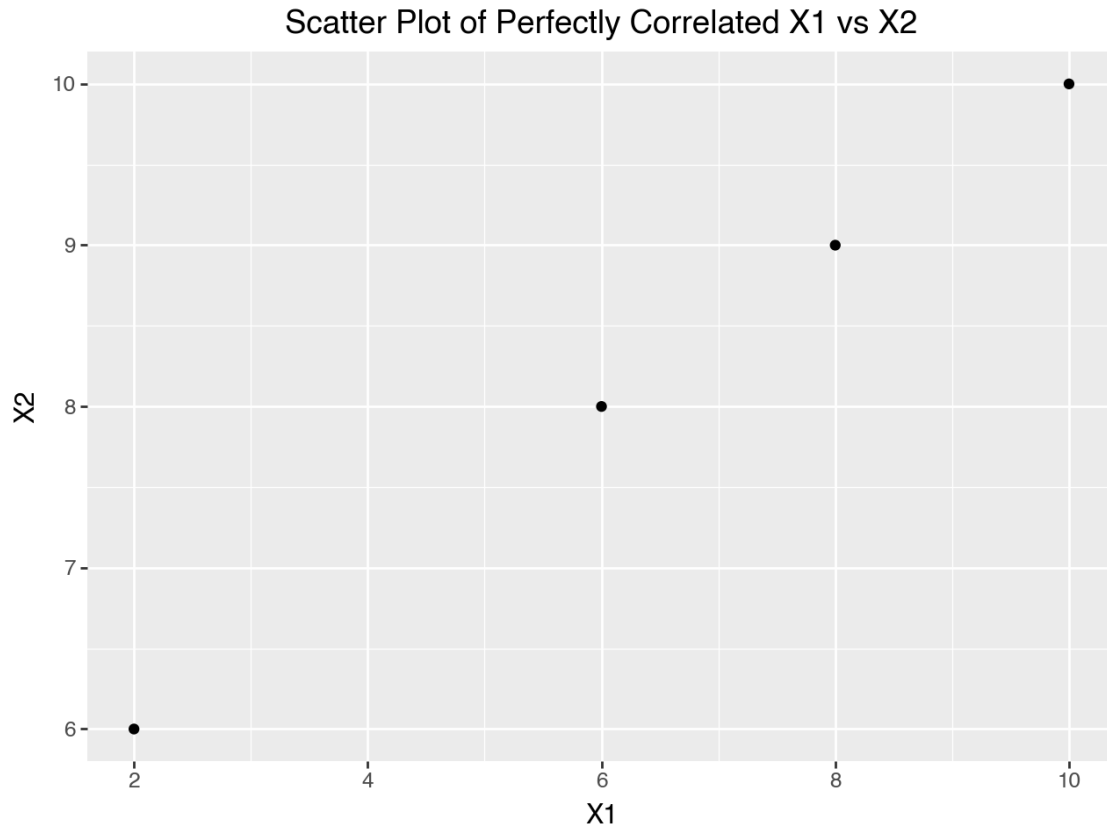
```
[10]:
```

	df	sum_sq	mean_sq	F	PR(>F)
X2	1.0	171.125	171.125000	4.127638	0.08846
Residual	6.0	248.750	41.458333	NaN	NaN

```
[11]: # Perfectly correlated case
      X1 = [2, 8, 6, 10]
      X2 = [6, 9, 8, 10]
      Y = [23, 83, 63, 103]

      df_perfect = pd.DataFrame({
          'X1': X1,
          'X2': X2,
          'Y': Y
      })

      # Plot X1 vs X2
      plot_perfect = (ggplot(df_perfect, aes(x='X1', y='X2')) +
                      geom_point() +
                      labs(x='X1', y='X2', title='Scatter Plot of Perfectly_
↪Correlated X1 vs X2'))
      plot_perfect
```



```
[12]: # Correlation between X1 and X2
correlation_perfect = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation_perfect}')
```

Correlation between X1 and X2: 1.0

```
[13]: X = np.column_stack([np.ones(len(X1)), X1, X2])
X
```

```
[13]: array([[ 1.,  2.,  6.],
           [ 1.,  8.,  9.],
           [ 1.,  6.,  8.],
           [ 1., 10., 10.]])
```

```
[14]: # Matrix operations

XtX = np.dot(X.T, X)
print(f'XtX Matrix:\n{XtX}')
```

XtX Matrix:

```
[[ 4.  26.  33.]
 [ 26. 204. 232.]
```

```
[ 33. 232. 281.]]
```

```
[15]: # Eigenvalues
eigenvalues = np.linalg.eigvals(XtX)
print(f'Eigenvalues:\n{eigenvalues}') #2nd eigen val very close to zero !!!!
#numerically near singular
#this is going to be an issue for us:
```

Eigenvalues:

```
[ 4.81365468e+02 -1.27267426e-15  7.63453185e+00]
```

```
[16]: import scipy.linalg as spla

# Attempt to invert the matrix
XtX_inv = spla.solve(XtX, np.eye(3))
print(f'Inverse of XtX:\n{XtX_inv}')

#it blows up and gives you a warning on the "Condition Number" of the matrix
```

Inverse of XtX:

```
[[-4.56937087e+15 -4.56937087e+14  9.13874175e+14]
 [-4.56937087e+14 -4.56937087e+13  9.13874175e+13]
 [ 9.13874175e+14  9.13874175e+13 -1.82774835e+14]]
```

```
/var/folders/8s/b3g9yw6125n1scdnf7737s_c0000gn/T/ipykernel_11576/3933071815.py:4
: LinAlgWarning: Ill-conditioned matrix (rcond=3.08324e-19): result may not be
accurate.
```

```
[17]: #the warning is basically telling you the result is nonsense. Sanity check!
```

```
[18]: XtX_inv @ XtX # this is not the identity.... cry
```

```
[18]: array([[ 2.75    , -2.    , 16.75   ],
 [ 0.21875,  6.75    ,  4.46875],
 [-1.    , -4.    , -4.    ]])
```

```
[19]: #If you use np.linalg.inv ... it doesn't do the float calcs exactly and will
      ↪ actually let you invert a singular matrix... watch out!
np.linalg.inv(XtX)

# Interesting discussion of the issue: https://github.com/numpy/numpy/issues/2074
      ↪ 2074
```

```
[19]: array([[ 2.25179981e+16,  2.25179981e+15, -4.50359963e+15],
 [ 2.25179981e+15,  2.25179981e+14, -4.50359963e+14],
 [-4.50359963e+15, -4.50359963e+14,  9.00719925e+14]])
```

1.1 Variance Inflation Factors - VIFs

```
[20]: # Variance Inflation - generating new data
np.random.seed(0)
X1 = np.random.uniform(25, 50, 100)
X2 = np.random.normal(10, 3, 100)
eps = np.random.normal(0, 4, 100)

df_vif = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'Y': 7 + 3 * X1 + 5 * X2 + eps
})

# Correlation between X1 and X2
correlation_X1_X2 = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation_X1_X2}')
```

Correlation between X1 and X2: -0.018247827215873588

```
[21]: # Linear model with X1 and X2
mod1 = ols('Y ~ X1 + X2', data=df_vif).fit()
mod1.summary()
```

[21]:

Dep. Variable:	Y	R-squared:	0.978
Model:	OLS	Adj. R-squared:	0.978
Method:	Least Squares	F-statistic:	2193.
Date:	Thu, 25 Sep 2025	Prob (F-statistic):	1.82e-81
Time:	10:51:30	Log-Likelihood:	-277.67
No. Observations:	100	AIC:	561.3
Df Residuals:	97	BIC:	569.2
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.9068	2.505	2.358	0.020	0.934	10.880
X1	2.9676	0.055	54.178	0.000	2.859	3.076
X2	5.1591	0.132	39.062	0.000	4.897	5.421

Omnibus:	0.196	Durbin-Watson:	2.109
Prob(Omnibus):	0.907	Jarque-Bera (JB):	0.026
Skew:	0.032	Prob(JB):	0.987
Kurtosis:	3.046	Cond. No.	247.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[22]: vif_model_X1o = ols('X1 ~ X2', data=df_vif).fit()
r2j_X1o = vif_model_X1o.rsquared
vif_j_x1 = 1/(1-r2j_X1o)
```



```
print(f'VIF for X1: {vif_j_x1}')
```

VIF for X1: 1.0003330941128432

1.1.1 VIF Close to 1 indicates not very much multicollinearity between X1 and other predictors.

1.1.2 On the other hand...

```
[23]: # Creating a nearly perfectly collinear variable X3
df_vif['X3'] = 2 * df_vif['X1'] + 3 * df_vif['X2'] + np.random.normal(0, 0.01, 100)

correlation_X1_X3 = np.corrcoef(df_vif['X1'], df_vif['X3'])[0, 1]
correlation_X2_X3 = np.corrcoef(df_vif['X2'], df_vif['X3'])[0, 1]
correlation_comb = np.corrcoef(2 * df_vif['X1'] + 3 * df_vif['X2'], df_vif['X3'])[0, 1]

print(f'Correlation between X1 and X3: {correlation_X1_X3}')
print(f'Correlation between X2 and X3: {correlation_X2_X3}')
print(f'Correlation between (2*X1 + 3*X2) and X3: {correlation_comb}')
```

Correlation between X1 and X3: 0.8464409676819133

Correlation between X2 and X3: 0.5169479332790722

Correlation between (2*X1 + 3*X2) and X3: 0.9999998570072173

```
[24]: # Linear model with X1, X2, and X3
mod2 = ols('Y ~ X1 + X2 + X3', data=df_vif).fit()
mod2.summary()
```

[24]:

Dep. Variable:	Y	R-squared:	0.979
Model:	OLS	Adj. R-squared:	0.978
Method:	Least Squares	F-statistic:	1469.
Date:	Thu, 25 Sep 2025	Prob (F-statistic):	4.67e-80
Time:	10:51:30	Log-Likelihood:	-276.92
No. Observations:	100	AIC:	561.8
Df Residuals:	96	BIC:	572.3
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.8079	2.501	2.322	0.022	0.844	10.772
X1	-102.9961	87.588	-1.176	0.243	-276.858	70.866
X2	-153.7766	131.375	-1.171	0.245	-414.553	107.000
X3	52.9822	43.795	1.210	0.229	-33.949	139.914

Omnibus:	0.280	Durbin-Watson:	2.107
Prob(Omnibus):	0.870	Jarque-Bera (JB):	0.044
Skew:	0.020	Prob(JB):	0.978
Kurtosis:	3.094	Cond. No.	4.73e+04

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.73e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
[25]: # VIF calculation for X1
vif_model_X1 = ols('X1 ~ X2 + X3', data=df_vif).fit()
r2j_X1 = vif_model_X1.rsquared
vif_j_X1 = 1 / (1 - r2j_X1)
print(f'VIF for X1: {vif_j_X1}')
```

VIF for X1: 2570128.137094504

```
[26]: # VIF calculation for X2
vif_model_X2 = ols('X2 ~ X1 + X3', data=df_vif).fit()
r2j_X2 = vif_model_X2.rsquared
vif_j_X2 = 1 / (1 - r2j_X2)
print(f'VIF for X2: {vif_j_X2}')
```

VIF for X2: 994491.2150962795

```
[27]: # VIF calculation for X3
vif_model_X3 = ols('X3 ~ X1 + X2', data=df_vif).fit()
r2j_X3 = vif_model_X3.rsquared
vif_j_X3 = 1 / (1 - r2j_X3)
print(f'VIF for X3: {vif_j_X3}')
```

VIF for X3: 3506271.326686703

1.1.3 VIFs»>10 indicate severe multicollinearity among predictors.

1.2 Exercise

For the dataset “multicollinearity.txt”, perform the following:

1. Check the pairwise correlations of the predictors.
2. Compare the coefficient for each predictor in two cases: a. when regressing Y on that predictor only, and b. when regressing Y on all predictors. Do you see a major change in the coefficients? Their standard errors? What does this tell you about the presence of multicollinearity?
3. Calculate the VIFs for each predictor. Which VIFs are concerning? Comment on what you find.
4. Consider regressing X4 onto the other predictors X1 through X3. Save the residuals from this model and call them `res4`. Then regress Y onto `res4` and look at the coefficient. Compare this coefficient to the coefficients for X4 you found in part 2. What do you notice?
5. Explain geometrically what we are doing in #4. Are we surprised by the results?

```
[28]: multicollinearity = pd.read_csv("/Users/lokeshmuvva/Documents/regression_f25/
↳data/multicollinearity.txt", sep='\s+')
print(multicollinearity.head())
```

```
Y = multicollinearity["y"]
X = multicollinearity[["x1", "x2", "x3", "x4"]]
```

	y	x1	x2	x3	x4
0	1.860	0.374	0.846	1.951	1.325
1	9.747	2.184	3.754	5.099	5.372
2	12.707	2.164	3.635	2.505	2.582
3	12.822	5.595	4.805	3.653	3.399
4	5.160	5.330	2.611	1.873	1.474

```
[29]: X.corr()
```

```
[29]:
```

	x1	x2	x3	x4
x1	1.000000	0.999811	0.999495	0.998843
x2	0.999811	1.000000	0.999671	0.999013
x3	0.999495	0.999671	1.000000	0.999306
x4	0.998843	0.999013	0.999306	1.000000

1. Checking the pairwise correlations indicates that there is severe collinearity among the predictors.

```
[30]: mc_model = ols('y ~ x1 + x2 + x3 + x4', data=multicollinearity).fit()
```

```
mc_model_x1 = ols('y ~ x1', data=multicollinearity).fit()
mc_model_x2 = ols('y ~ x2', data=multicollinearity).fit()
mc_model_x3 = ols('y ~ x3', data=multicollinearity).fit()
mc_model_x4 = ols('y ~ x4', data=multicollinearity).fit()

params_of_interest = ['x1', 'x2', 'x3', 'x4']

print("Coefficients when regressing Y on all variables ± SE:")
for param in params_of_interest:
    coef = mc_model.params[param]
    std_err = mc_model.bse[param]
    print(f"{param}: {coef:.4f} ± {std_err:.4f}")

print("\nCoefficients when regressing Y on that variable only ± SE:")
# Method 1: Individual models one by one
models = [mc_model_x1, mc_model_x2, mc_model_x3, mc_model_x4]
for i, model in enumerate(models):
    param = params_of_interest[i]
    coef = model.params[param]
    std_err = model.bse[param]
    print(f"{param}: {coef:.4f} ± {std_err:.4f}")
```

Coefficients when regressing Y on all variables ± SE:

x1: -0.1675 ± 0.2041

x2: 1.1364 ± 0.2806

x3: 0.9788 ± 0.2556

x4: 1.1175 ± 0.2104

Coefficients when regressing Y on that variable only ± SE:

x1: 2.1250 ± 0.0057

x2: 2.3613 ± 0.0053

x3: 2.9492 ± 0.0063

x4: 4.2002 ± 0.0116

2. There is also a distinct change in the increase/switching of signs in the coefficients, as well as the big difference in standard errors, further indicating the presence of multicollinearity.

```
[31]: def calculate_VIF(r2_j):  
        return (1/(1-r2_j))  
  
        # VIF calculation for X1  
vif_model_X1 = ols('x1 ~ x2 + x3 + x4', data=multicollinearity).fit()  
r2j_X1 = vif_model_X1.rsquared  
vif_j_X1 = calculate_VIF(r2j_X1)  
print(f'VIF for X1: {vif_j_X1}')  
  
        # VIF calculation for X2  
vif_model_X2 = ols('x2 ~ x1 + x3 + x4', data=multicollinearity).fit()  
r2j_X2 = vif_model_X2.rsquared  
vif_j_X2 = calculate_VIF(r2j_X2)  
print(f'VIF for X2: {vif_j_X2}')  
  
        # VIF calculation for X3  
vif_model_X3 = ols('x3 ~ x1 + x2 + x4', data=multicollinearity).fit()  
r2j_X3 = vif_model_X3.rsquared  
vif_j_X3 = calculate_VIF(r2j_X3)  
print(f'VIF for X3: {vif_j_X3}')  
  
        # VIF calculation for X4  
vif_model_X4 = ols('x4 ~ x1 + x2 + x3', data=multicollinearity).fit()  
r2j_X4 = vif_model_X4.rsquared  
vif_j_X4 = calculate_VIF(r2j_X4)  
print(f'VIF for X4: {vif_j_X4}')
```

VIF for X1: 2653.8686128804293

VIF for X2: 4066.1980032159418

VIF for X3: 2162.4583519023327

VIF for X4: 722.2889185367274

3. All VIFs are concerning as all values are all much much greater than 10, indicating severe multicollinearity. This matches with the results seen in 1 and 2.

```
[32]: res4_model = ols('x4 ~ x1 + x2 + x3', data=multicollinearity).fit()  
multicollinearity['res4'] = res4_model.resid  
print(multicollinearity.head())
```

	y	x1	x2	x3	x4	res4
0	1.860	0.374	0.846	1.951	1.325	-0.089034
1	9.747	2.184	3.754	5.099	5.372	1.792657
2	12.707	2.164	3.635	2.505	2.582	0.724258
3	12.822	5.595	4.805	3.653	3.399	0.727658
4	5.160	5.330	2.611	1.873	1.474	0.033488

```
[33]: y_reg_model = ols('y ~ res4', data=multicollinearity).fit()
print(f"Res4 coefficient (when regressing Y on this predictor only): \u2192
      \u2192{y_reg_model.params['res4']}")
print(f"X4 coefficient (when regressing Y on all predictors): {mc_model.
      \u2192params['x4']}")
print(f"X4 coefficient (when regressing Y on this predictor only): {mc_model_x4.
      \u2192params['x4']}")
```

Res4 coefficient (when regressing Y on this predictor only): 1.1175308696447717

X4 coefficient (when regressing Y on all predictors): 1.1175308696429855

X4 coefficient (when regressing Y on this predictor only): 4.2002489226563

4. The coefficient for Res4 is almost identical to the coefficient of X4 when Y is regressed on all the variables, to the 11th decimal place.
5. The residuals from $x4 \sim x1 + x2 + x3$ represent the part of $x4$ that is orthogonal to (uncorrelated with) $x1$, $x2$, and $x3$. In the multiple regression $y \sim x1 + x2 + x3 + x4$, the coefficient on $x4$ captures the relationship between y and $x4$ after controlling for $x1$, $x2$, and $x3$, which is what the residuals represent. The stronger the multicollinearity, the smaller the residuals from $x4 \sim x1 + x2 + x3$ become, the larger the standard errors become in the original regression, but the coefficient stays the same by the FWL (Frisch-Waugh-Lovell) theorem.

```
[34]: # Check how much of x4 is explained by the other variables
r_squared_x4 = res4_model.rsquared
print(f"R\u00b2 when regressing x4 on x1,x2,x3: {r_squared_x4:.4f}")
print(f"Variance in x4 explained by other predictors: {r_squared_x4*100:.1f}%")

# The residuals have much less variance than original x4
print(f"Variance of original x4: {multicollinearity['x4'].var():.4f}")
print(f"Variance of res4: {multicollinearity['res4'].var():.4f}")
```

R² when regressing x4 on x1,x2,x3: 0.9986

Variance in x4 explained by other predictors: 99.9%

Variance of original x4: 855.7557

Variance of res4: 1.1848